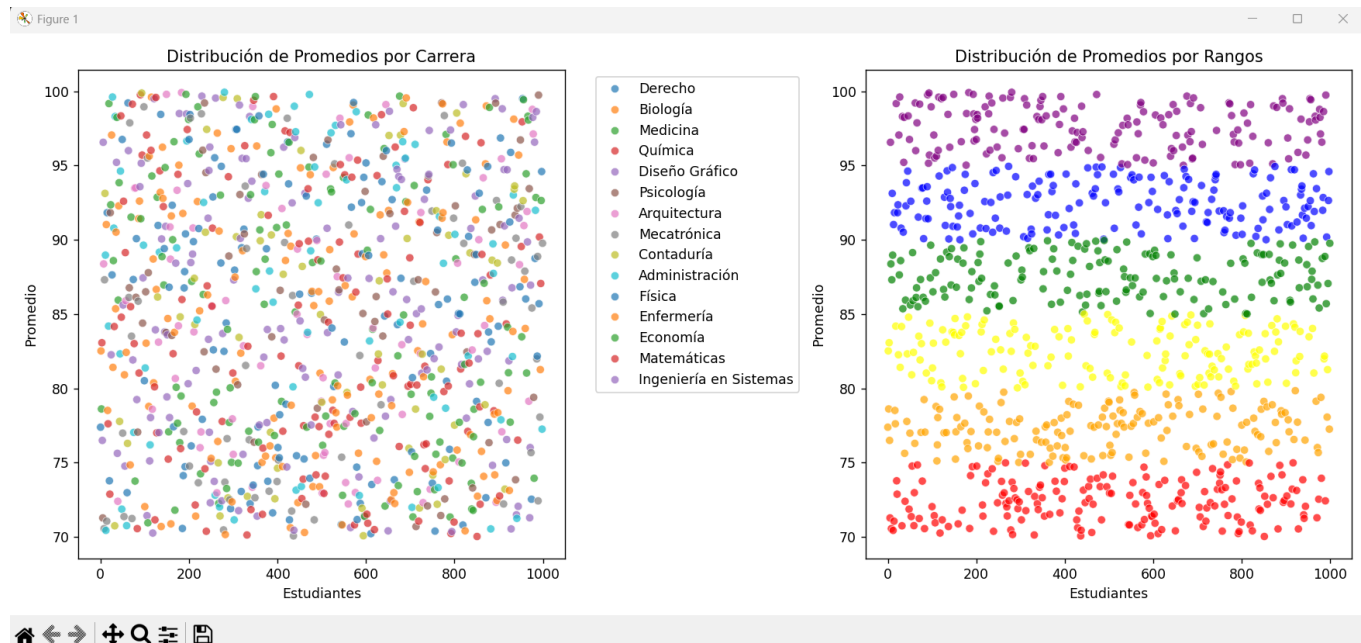


Práctica 2-3

Base de datos con 1000 alumnos y graficado.



By Josue C. Saucedo, *Instituto Tecnológico Superior de Irapuato*

EN esta práctica se realizó un análisis de los promedios de estudiantes agrupados por carrera y rangos de calificación. El objetivo fue generar datos ficticios de estudiantes, almacenarlos en una base de datos MySQL, y visualizar la distribución de los promedios utilizando gráficos de dispersión. Se utilizaron herramientas como Python, MySQL, Pandas, Matplotlib y Seaborn para la manipulación de datos y la generación de gráficos. Además, se implementó una función para asignar colores específicos a diferentes rangos de promedios, lo que permitió una visualización más clara y detallada de los datos.

OBJETIVOS

El objetivo principal de este proyecto es generar una base de datos de estudiantes de manera aleatoria, almacenarla en MySQL y posteriormente analizar los datos utilizando Pandas. Además, se busca visualizar la distribución de los promedios de los estudiantes agrupados por carrera y por rangos de calificaciones.

MATERIALES

- Python 3
- MySQL Server
- Librerías de Python: pymysql, pandas, numpy, matplotlib, seaborn, faker
- Un entorno de desarrollo (Jupyter Notebook, VS Code, PyCharm, etc.)

DESARROLLO DE LA PRACTICA

A continuación, se describe paso a paso el desarrollo de la práctica:

1. **Creación de la base de datos y tabla en MySQL**
 - Se establece una conexión con MySQL y se crea una base de datos llamada escuela.
 - Se define la estructura de la tabla alumnos con los campos especificados.
2. **Generación de datos aleatorios**
 - Se usa la librería Faker para generar nombres y apellidos aleatorios.
 - Se asigna una de las 15 carreras de manera aleatoria a cada estudiante.
 - Se genera un promedio entre 70 y 100, determinando si el alumno es egresado.
3. **Almacenamiento en MySQL**
 - Se insertan los 1000 registros generados en la base de datos.

4. Lectura de datos con Pandas

- Se recuperan los datos almacenados y se transforman en un DataFrame para su análisis.

5. Visualización de datos

- Se crean dos gráficos de dispersión:
 - Uno con la distribución de promedios agrupados por carrera.
 - Otro con la distribución de promedios por rangos de calificaciones (70-85, 85-90, 90-100).

CODIGO

```
import pymysql

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from faker import Faker

# Configuración de conexión a MySQL
DB_HOST = "localhost"
DB_USER = "root"
DB_PASSWORD = "12345678"
DB_NAME = "COLEGIO"
TABLE_NAME = "alumnos"

# Crear conexión a MySQL y base de datos
connection = pymysql.connect(host=DB_HOST,
                             user=DB_USER, password=DB_PASSWORD)
cursor = connection.cursor()
cursor.execute(f"CREATE DATABASE IF NOT EXISTS {DB_NAME}")
connection.select_db(DB_NAME)

# Crear tabla
cursor.execute(f"""
```

```

CREATE TABLE IF NOT EXISTS {TABLE_NAME}
(
    id INT AUTO_INCREMENT PRIMARY KEY,
    numero_control VARCHAR(10) UNIQUE,
    nombre VARCHAR(50),
    apellido_paterno VARCHAR(50),
    apellido_materno VARCHAR(50),
    carrera VARCHAR(50),
    promedio FLOAT,
    egresado BOOLEAN
)
)
)

# Generar datos ficticios
faker = Faker("es_MX")
carreras = ["Ingeniería en Sistemas",
"Administración", "Contaduría",
"Psicología", "Derecho", "Medicina",
"Arquitectura", "Biología", "Física",
"Matemáticas", "Diseño Gráfico", "Economía",
"Mecatrónica", "Química", "Enfermería"]
datos = []

for _ in range(1000):
    numero_control =
    faker.unique.random_number(digits=8,
fix_len=True)
    nombre = faker.first_name()
    apellido_paterno = faker.last_name()
    apellido_materno = faker.last_name()
    carrera = np.random.choice(carreras)
    promedio = round(np.random.uniform(70,
100), 2)
    egresado = promedio >= 70
    datos.append((numero_control, nombre,
apellido_paterno, apellido_materno, carrera,
promedio, egresado))

# Insertar datos en MySQL
try:
    query = f"INSERT INTO {TABLE_NAME}
(numero_control, nombre, apellido_paterno,
apellido_materno, carrera, promedio,
egresado) VALUES (%s, %s, %s, %s, %s, %s,
%s)"

```

```

cursor.executemany(query, datos)
connection.commit()
print(f"{len(datos)} registros insertados
correctamente.")
except Exception as e:
    print("Error al insertar datos:", e)
    connection.rollback()

# Contar el número de registros en la tabla
cursor.execute(f"SELECT COUNT(*) FROM
{TABLE_NAME}")
total_registros = cursor.fetchone()[0]
print(f"Total de registros en la tabla
'{TABLE_NAME}': {total_registros}")

# Leer datos con Pandas
df = pd.read_sql(f"SELECT * FROM {TABLE_NAME}",
connection)

# Verificar si se leyeron los datos correctamente
print(df.head()) # Imprimir las primeras filas
del DataFrame para verificar los datos

# Cerrar la conexión a la base de datos
connection.close()

# Limitar el DataFrame a los primeros 1000
registros
df_limited = df.head(1000)

# Configurar los subgráficos
plt.figure(figsize=(14, 6))

# Gráfico de dispersión agrupado por carrera
(Primer subgráfico)
plt.subplot(1, 2, 1) # 1 fila, 2 columnas,
gráfico 1
sns.scatterplot(data=df_limited,
x=df_limited.index, y="promedio", hue="carrera",
palette="tab10", alpha=0.7)
plt.title("Distribución de Promedios por
Carrera")
plt.xlabel("Estudiantes")
plt.ylabel("Promedio")

```

```
plt.legend(bbox_to_anchor=(1.05, 1),
loc='upper left')

# Gráfico de dispersión por rangos de
calificación (Segundo subgráfico)
plt.subplot(1, 2, 2) # 1 fila, 2 columnas,
gráfico 2

# Asegurarse de que solo se muestren 1000
alumnos
student_indices = np.arange(len(df_limited))

# Función para asignar colores según el
rango de promedios
def get_color(promedio):
    if 70 <= promedio < 75:
        return "red" # Color para el rango
70-75
    elif 75 <= promedio < 80:
        return "orange" # Color para el
rango 75-80
    elif 80 <= promedio < 85:
        return "yellow" # Color para el
rango 80-85
    elif 85 <= promedio < 90:
        return "green" # Color para el
rango 85-90
    elif 90 <= promedio < 95:
        return "blue" # Color para el rango
90-95
    elif 95 <= promedio <= 100:
        return "purple" # Color para el
rango 95-100
    else:
        return "gray" # Color por defecto
(no debería ocurrir)

# Aplicar colores según el promedio
colors =
df_limited["promedio"].apply(get_color)

# Graficar solo los primeros 1000
estudiantes
```

```
sns.scatterplot(x=student_indices,
y=df_limited["promedio"], c=colors, alpha=0.7)
plt.title("Distribución de Promedios por Rangos")
plt.xlabel("Estudiantes")
plt.ylabel("Promedio")

# Mostrar ambos gráficos
plt.tight_layout() # Ajusta el diseño para que
los gráficos no se superpongan
plt.show()
```

RESULTADOS

Tras la implementación del código, se obtuvieron los siguientes resultados:

- Se generaron y almacenaron correctamente los 1000 registros en la base de datos MySQL.
- La lectura de datos con Pandas fue exitosa y permitió una exploración rápida de los datos.
- Los dos gráficos generados permiten observar la distribución de promedios por carrera y por rangos de calificación, facilitando el análisis visual.

CONCLUSIÓN

El uso combinado de MySQL y Pandas proporciona una solución efectiva para almacenar y analizar grandes volúmenes de datos. La generación aleatoria de registros permite simular bases de datos realistas para pruebas y estudios estadísticos. Además, la visualización con Matplotlib y Seaborn facilita la interpretación de patrones y tendencias en los datos.

SELECT * FROM colegio_alumnos;

ID	numero_control	nombre	apellido_paterno	apellido_materno	carrera	promedio	egresado
992	72193935	Mayra	Ceballos	Carriz	Química	85.7	1
993	94004129	Ruben	Arriaga	Uribe	Química	90.19	1
994	80013596	Carmen	Calderon	Piñero	Meccatronica	76.06	1
995	26838584	Oliver	Benitez	Genaro	Enfermeria	81.26	1
996	38919136	Pascual	Ortiz	Cardenas	Química	87.06	1
997	80039435	Francisco	Vargas	Pedroza	Medicina	92.65	1
998	38959630	Alfonso	Garcia	Rangel	Física	94.6	1
999	61801398	Margarita	Valenzuela	Higuera	Administración	77.23	1
1000	80072290	Eva	Cruz	Torres	Neurología	89.77	1
2028	2028	2028	2028	2028	2028	2028	2028